IAM policy mishaps:

A cautionary tale of cloud misconfigurations

aws
COMMUNITY DAY
CEE





Agenda

- Intro
- IAM Policies 101
- Case 1: S3
- Case 2: SNS
- Case 3: Github Actions Federation
- Conclusions



$ aws sts get-caller-identity --profile unicrons_cloud





Samuel

Cloud Security Engineer @ flywire

sbldevnet

Andoni

Cloud Security Engineer @ PROWLER

andoniaf





Intro

Least privilege principle (POLP)

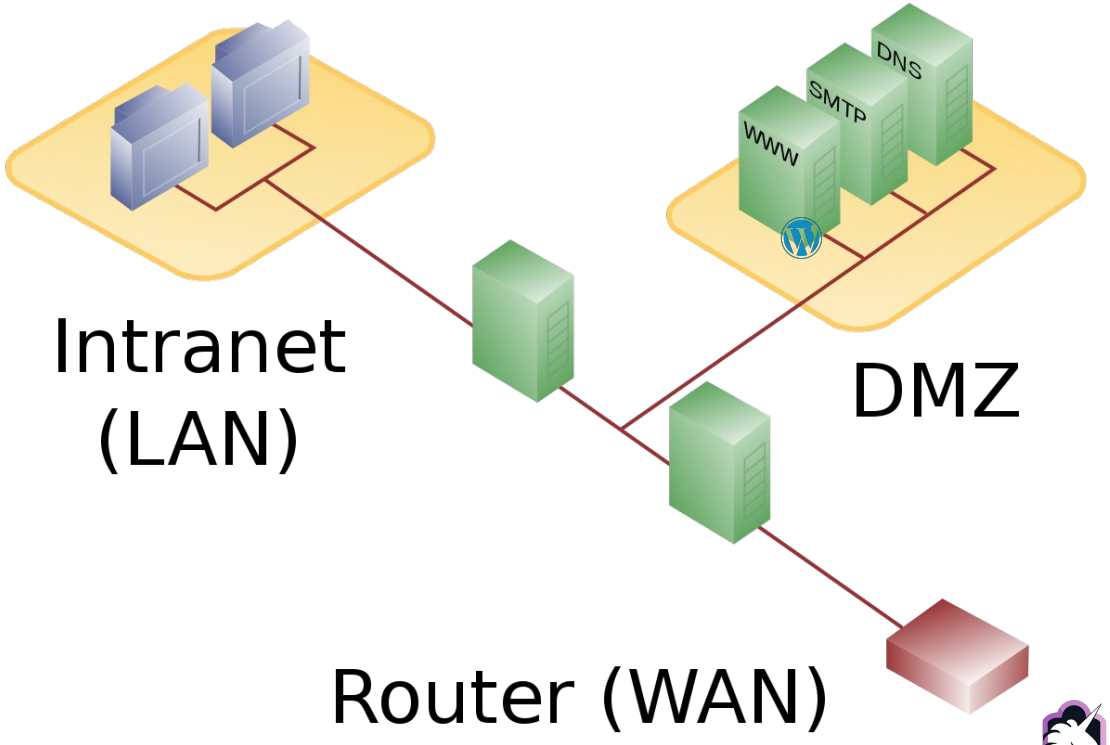
“is a computer security concept and practice that gives users limited access rights based on the tasks necessary to their job.”



DALL·E generated

Intro

“Let’s deploy Wordpress!”

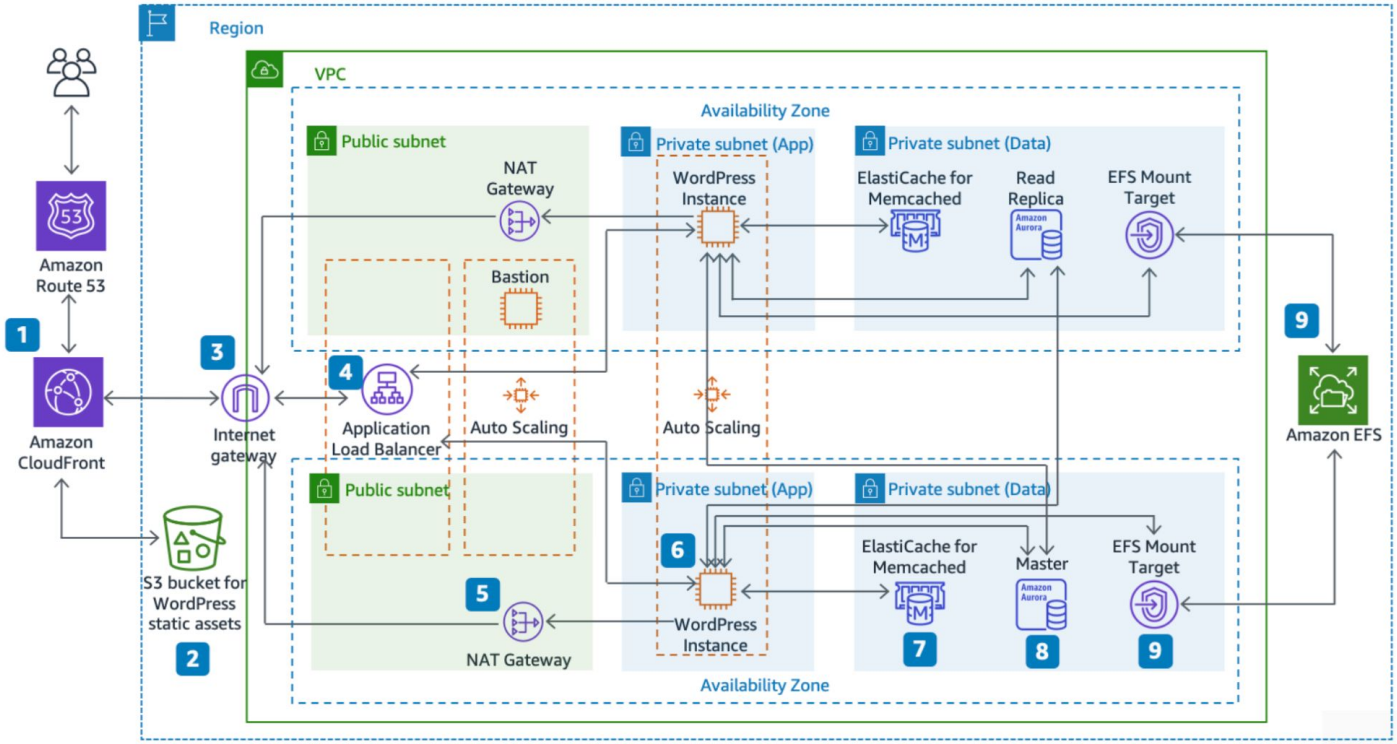


https://en.m.wikipedia.org/wiki/File:DMZ_network_diagram_2_firewall.svg

Intro

Best Practices for WordPress on AWS

AWS Whitepaper

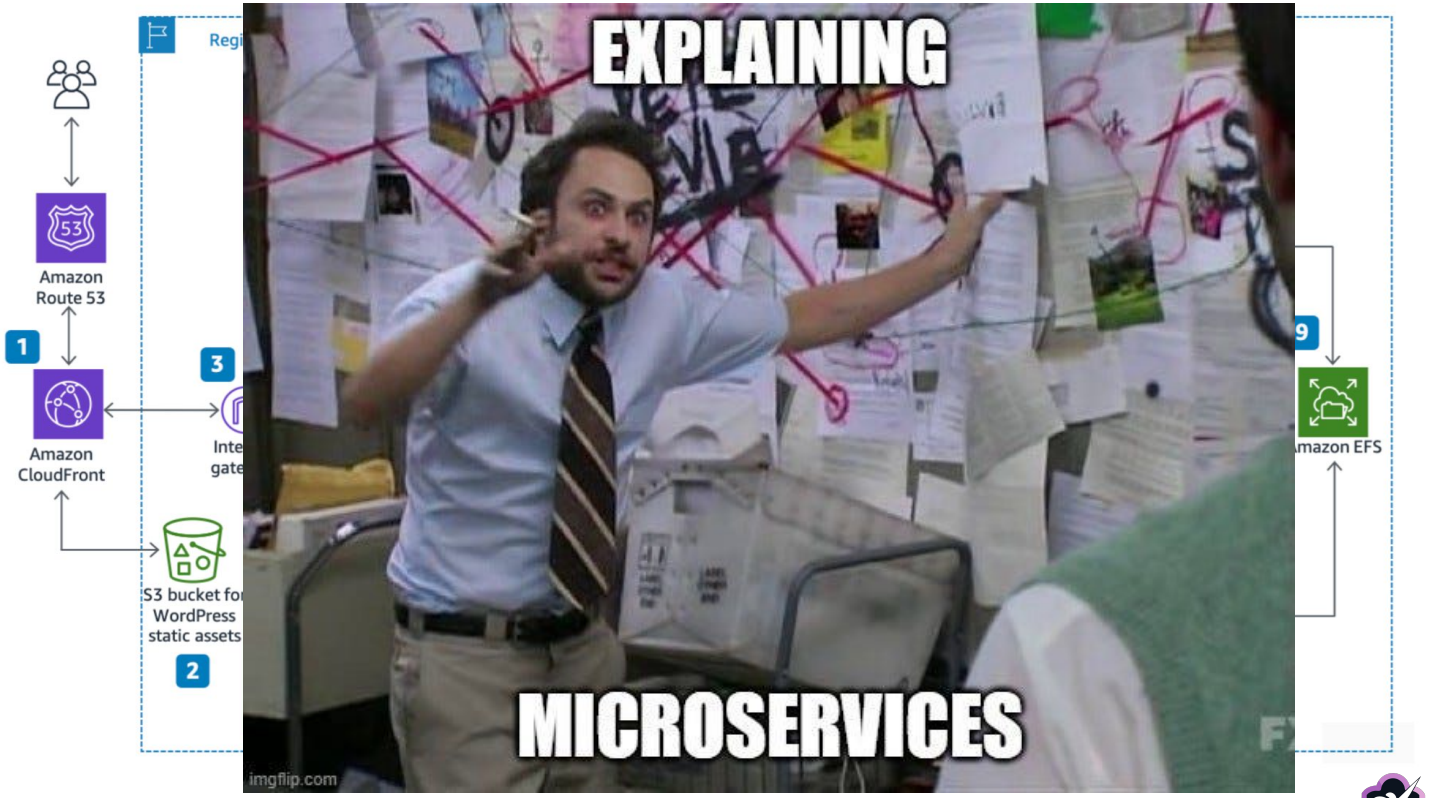


https://docs.aws.amazon.com/whitepapers/latest/best-practices-wordpress/reference-architecture.html

Intro

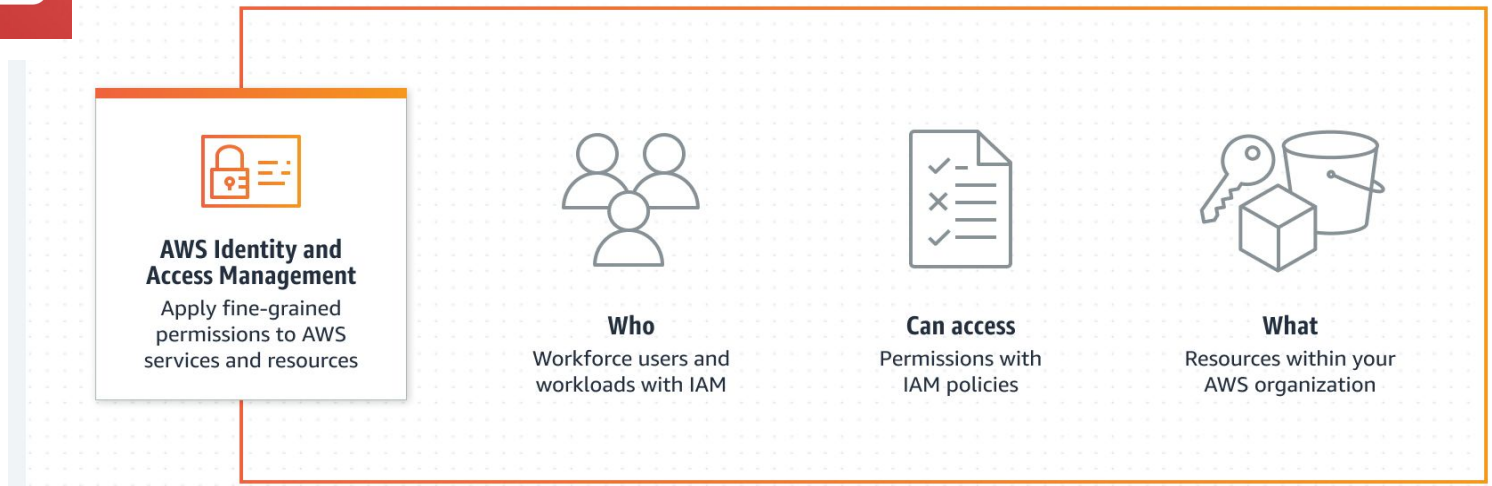
Best Practices for WordPress on AWS

AWS Whitepaper



https://docs.aws.amazon.com/whitepapers/latest/best-practices-wordpress/reference-architecture.html

Intro: IAM



https://aws.amazon.com/iam/

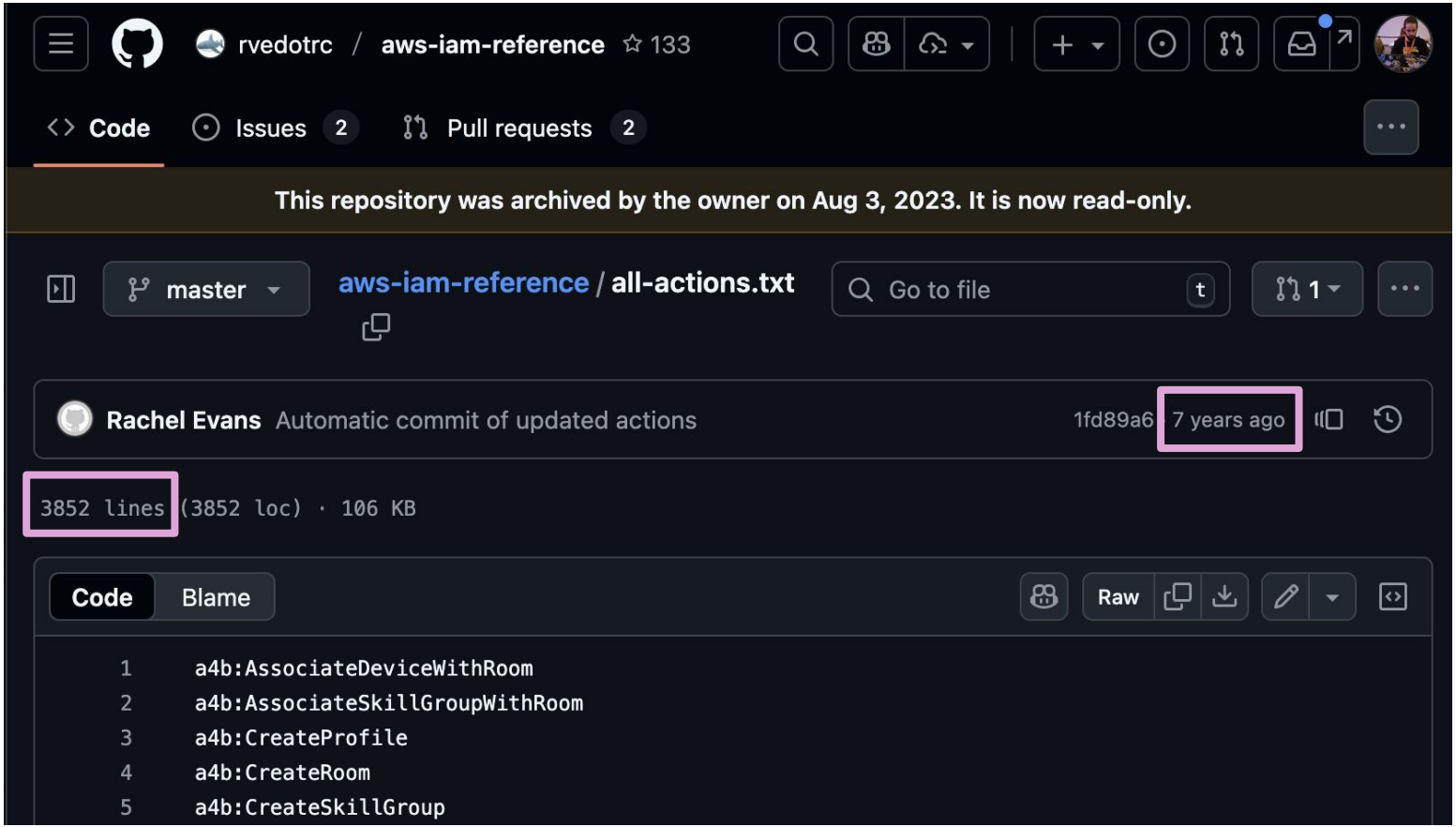
Intro: IAM

Allows read-only access to the IAM console

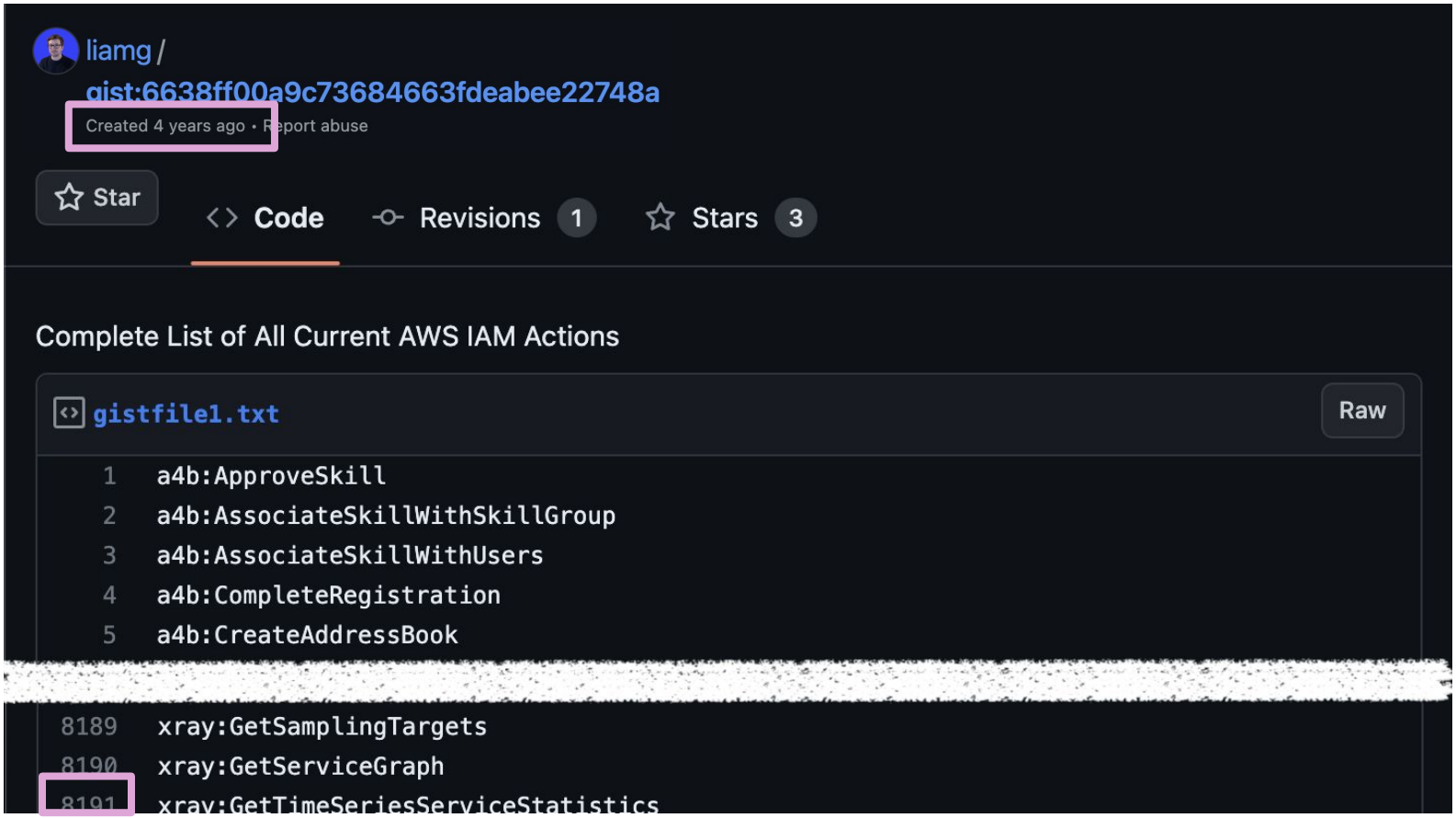
```
{
    "Version": "2012-10-17",
    "Statement": {
        "Effect": "Allow",
        "Action": [
            "iam:Get*",
            "iam:List*",
            "iam:Generate*"
        ],
        "Resource": "*"
    }
}
```



Intro: IAM actions



Intro: IAM actions



Intro: IAM actions

awsles / AwsServices Public

Notifications | Fork 8 | Star 34

Code | Issues | Pull requests | Actions | Projects | Security | ...

master | AwsServices / AwsServiceActions.

b8fc923 · 3 months ago

Jul 15, 2025, 7:52 PM GMT+2

awsles Update findings

3.66 MB

Code | Blame | Raw

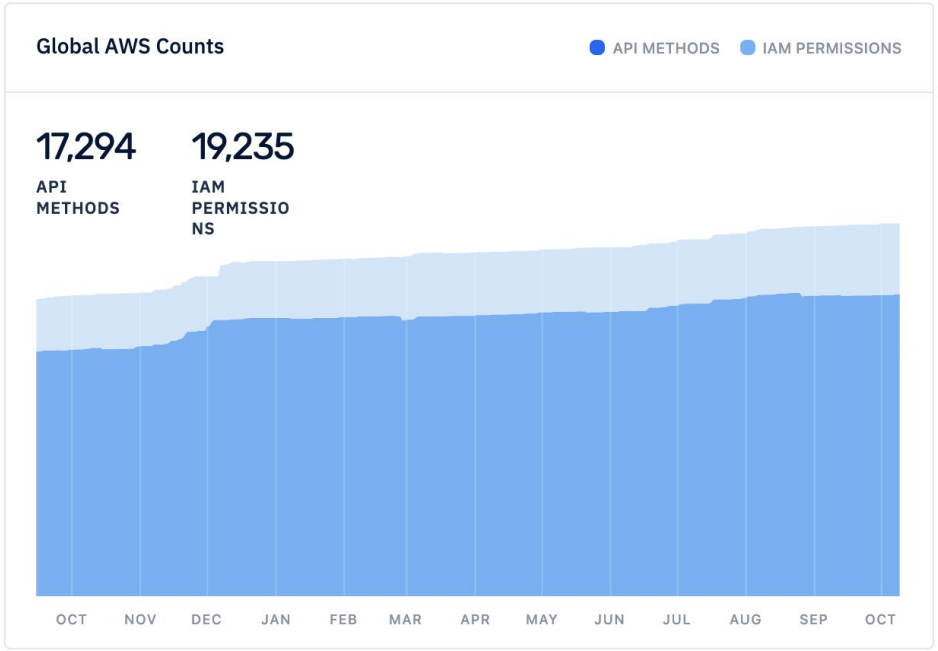
View raw

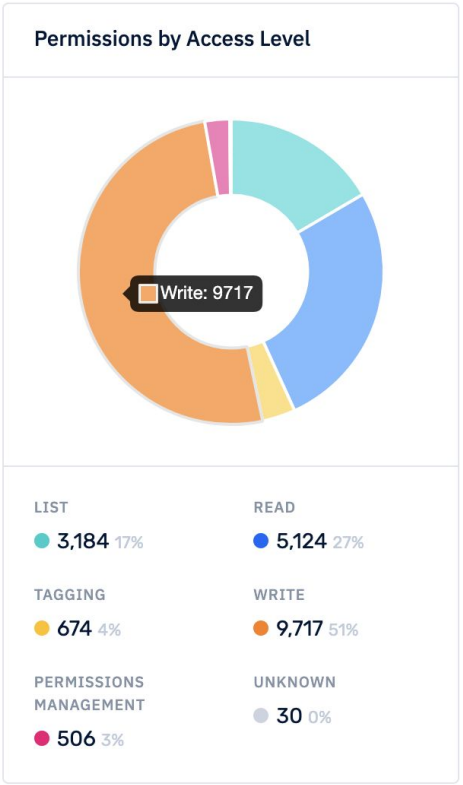
(Sorry about that, but we can't show files that are this big right now.)



```
~/test_ws/ AwsServices  master
wc -l  AwsServiceActions.txt
18913 AwsServiceActions.txt
```

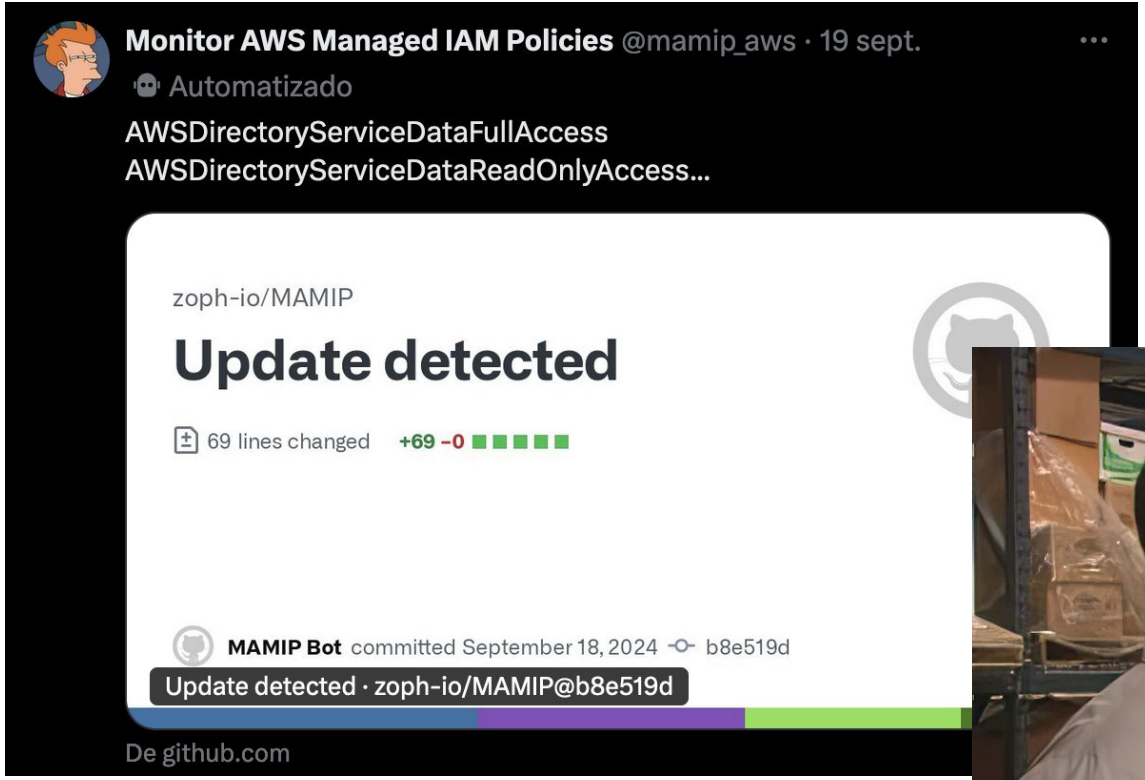
Intro: IAM actions





https://aws.permissions.cloud/

Intro: IAM actions



zoph-io / MAMIP

z0ph Victor GRENU



Intro: IAM actions

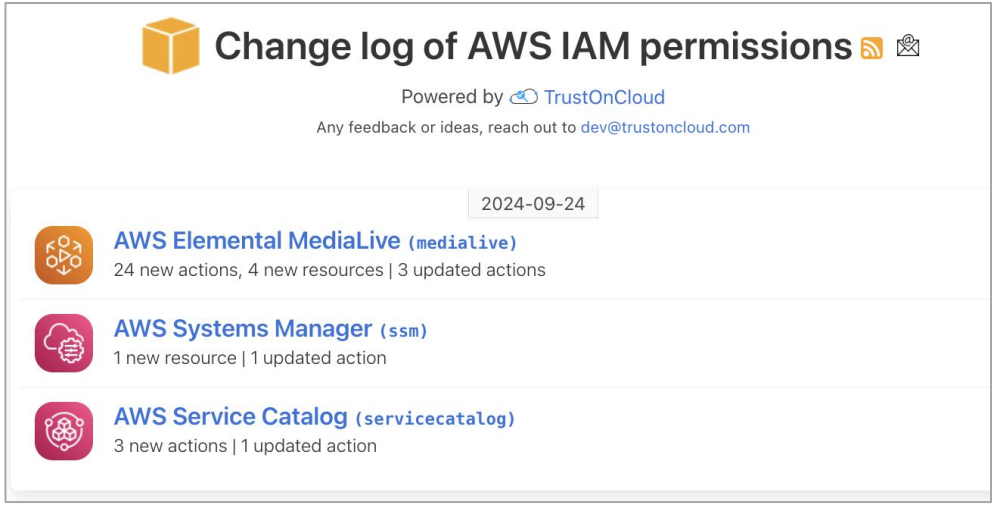
🤖 Dessert

Dessert is made by robots, for those that enjoy the industrial content.

🧁 IAM permission changes

- ds
- connect
- ds-data
- lambda
- amplifybackend
- s3express
- iot
- lambda
- ds
- personalize

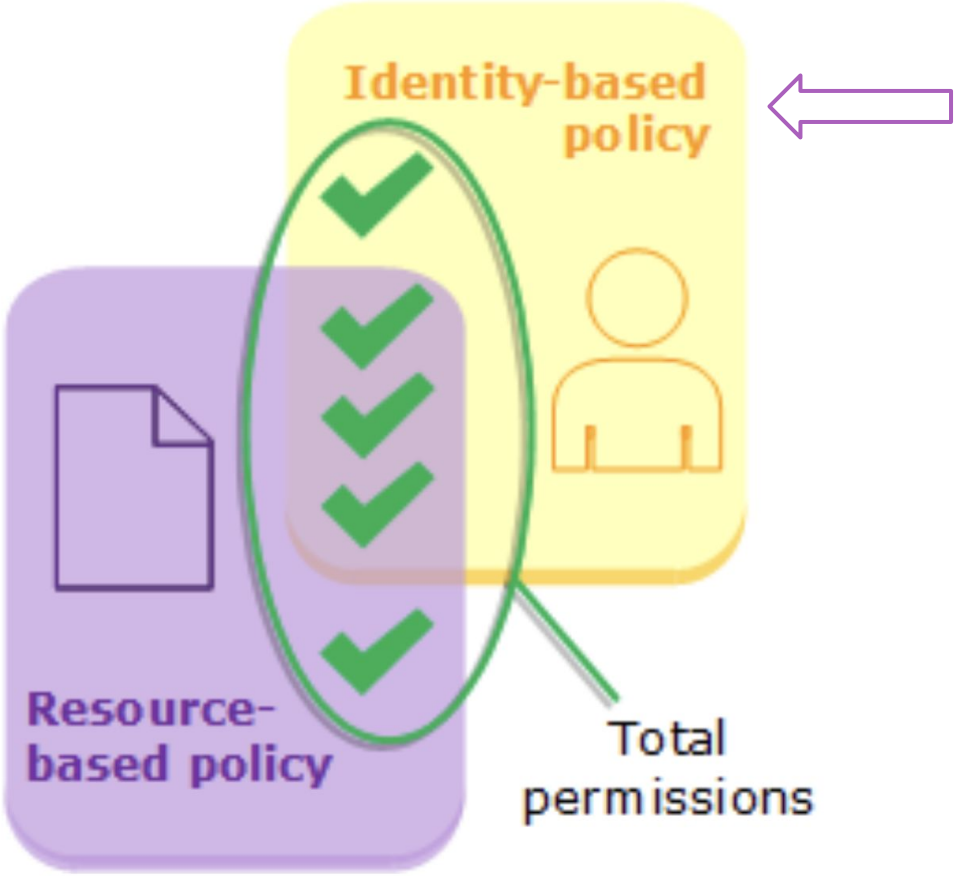
https://awssecuritydigest.com/



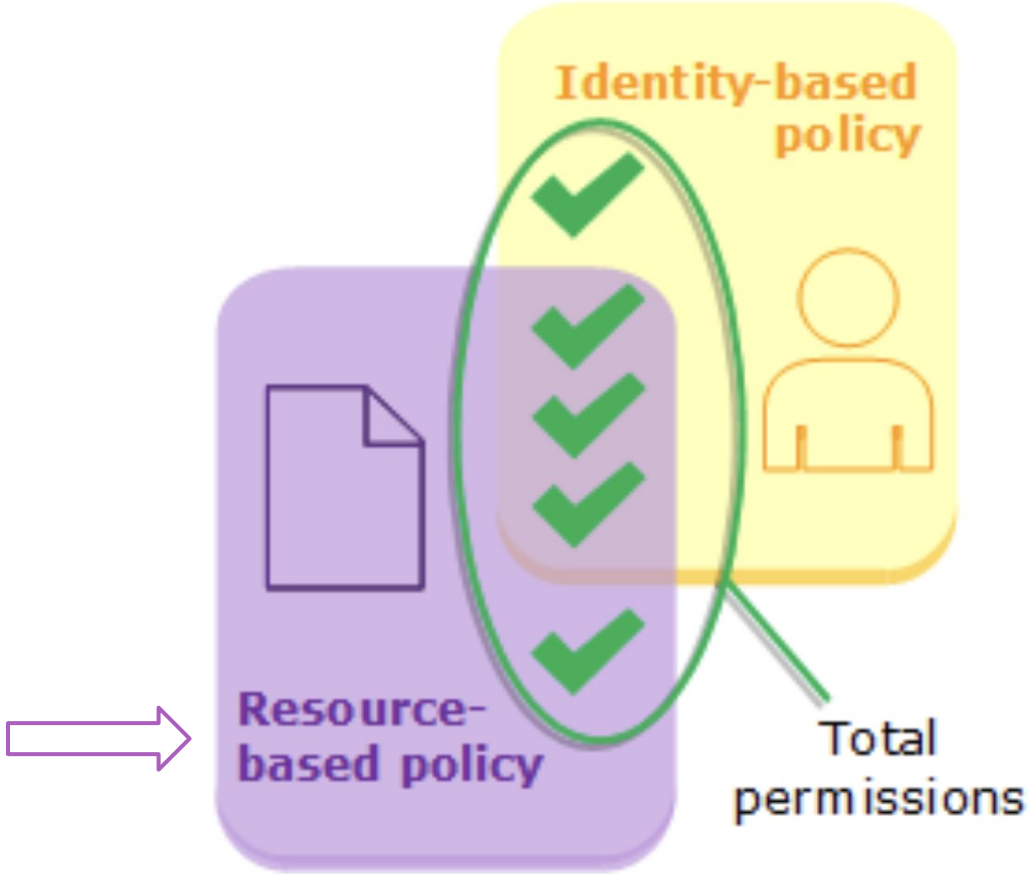
IAM Policies 101



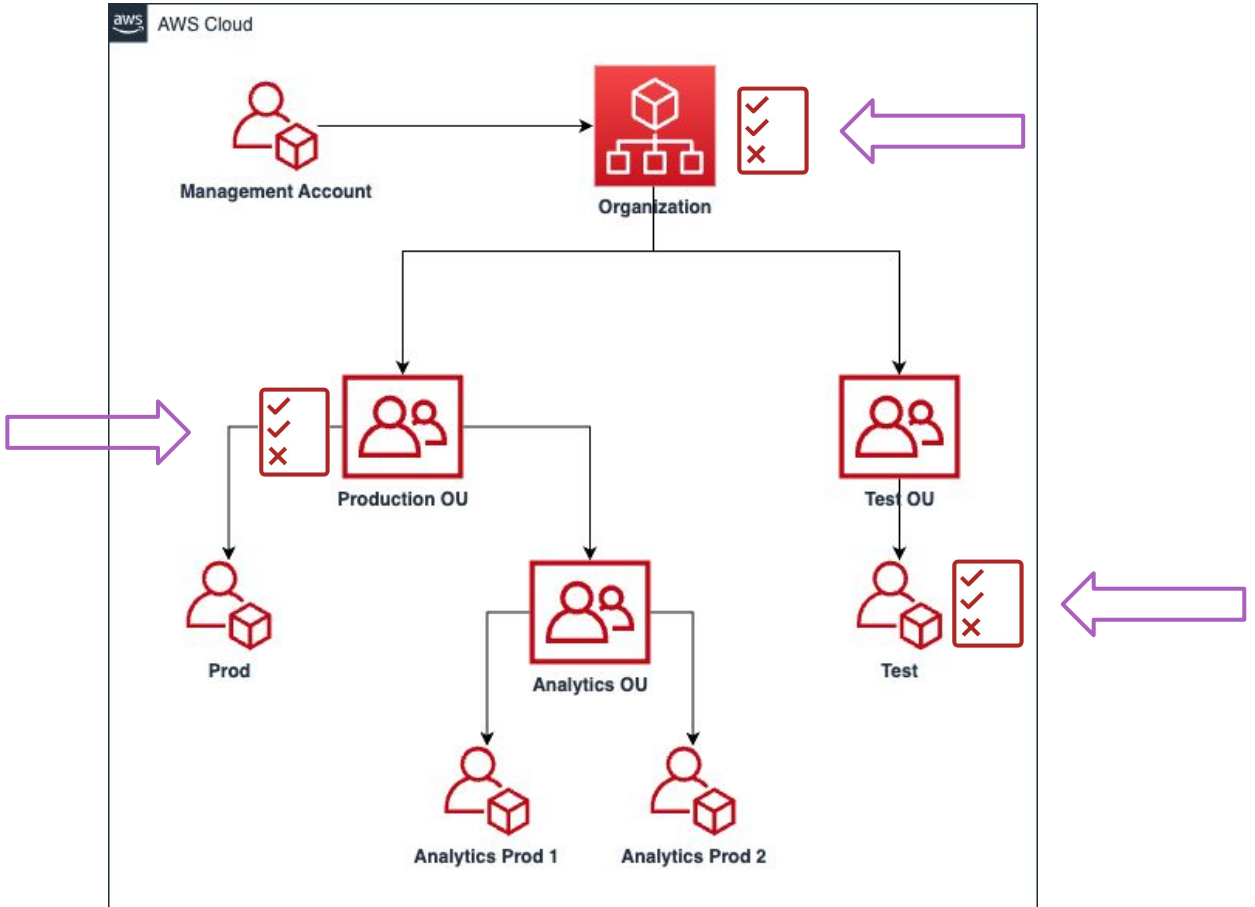
IAM Policies 101: Identity-based



IAM Policies 101: Resource-based



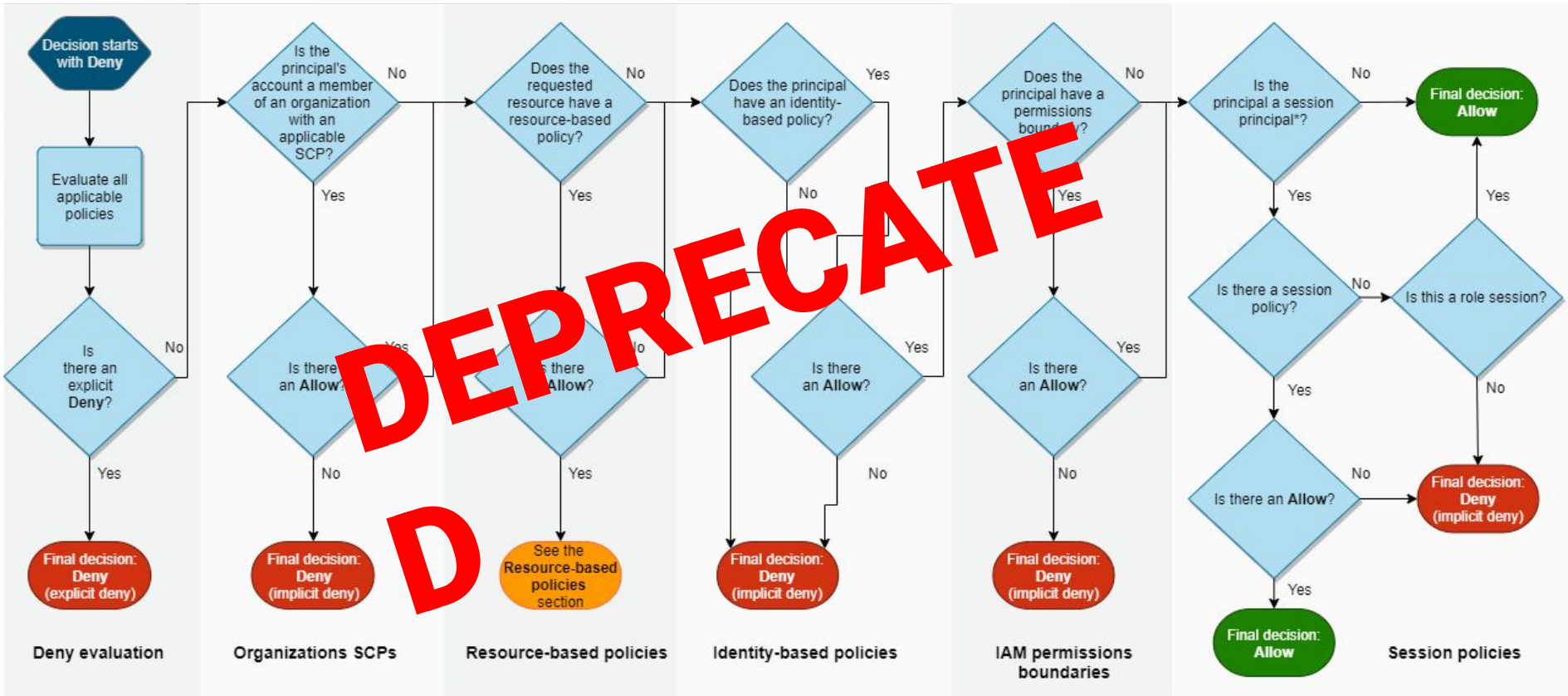
IAM Policies 101: Organizations



IAM Policies 101: Organizations

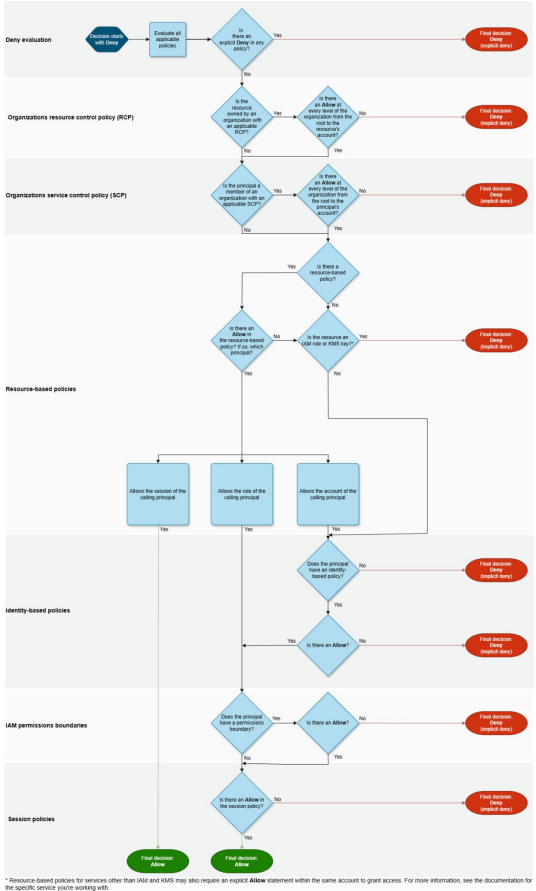


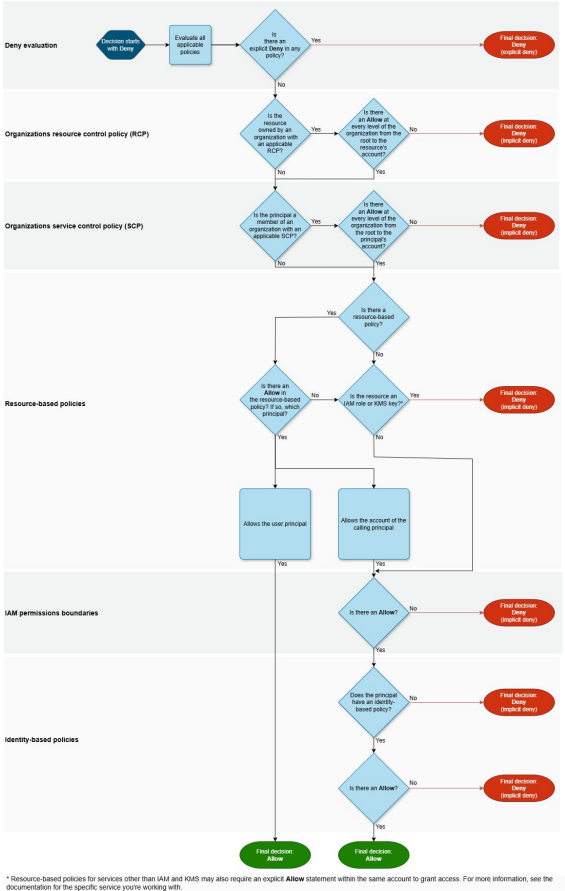
IAM Policies 101: Evaluation logic

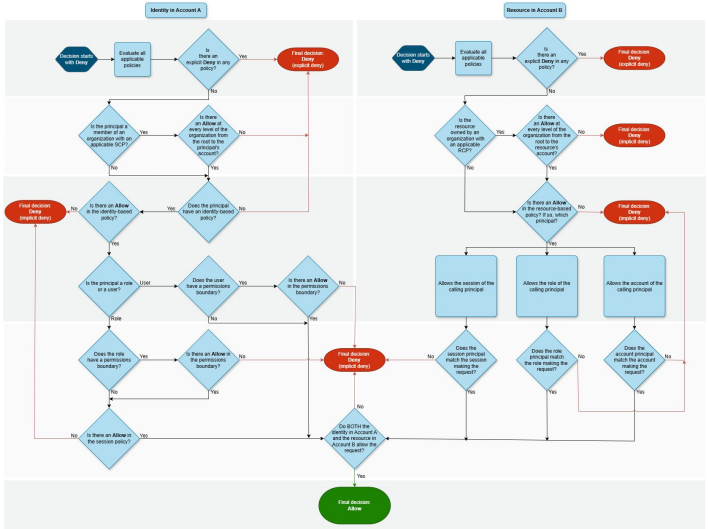


*A session principal is either a role session or an IAM federated user session.

IAM Policies 101: Evaluation logic





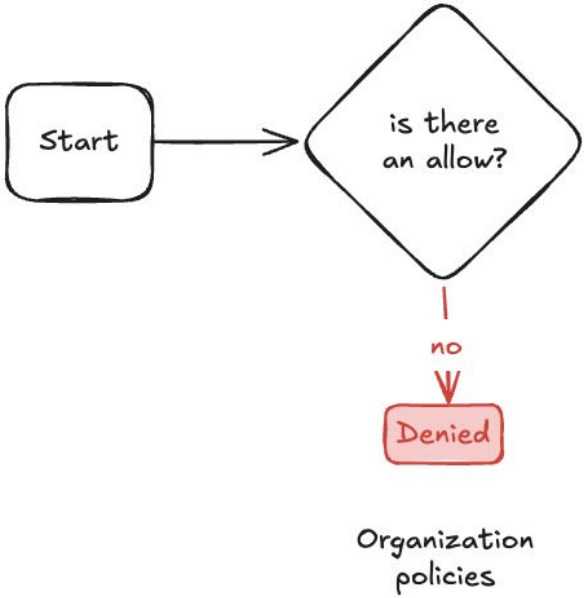


AWS Docs

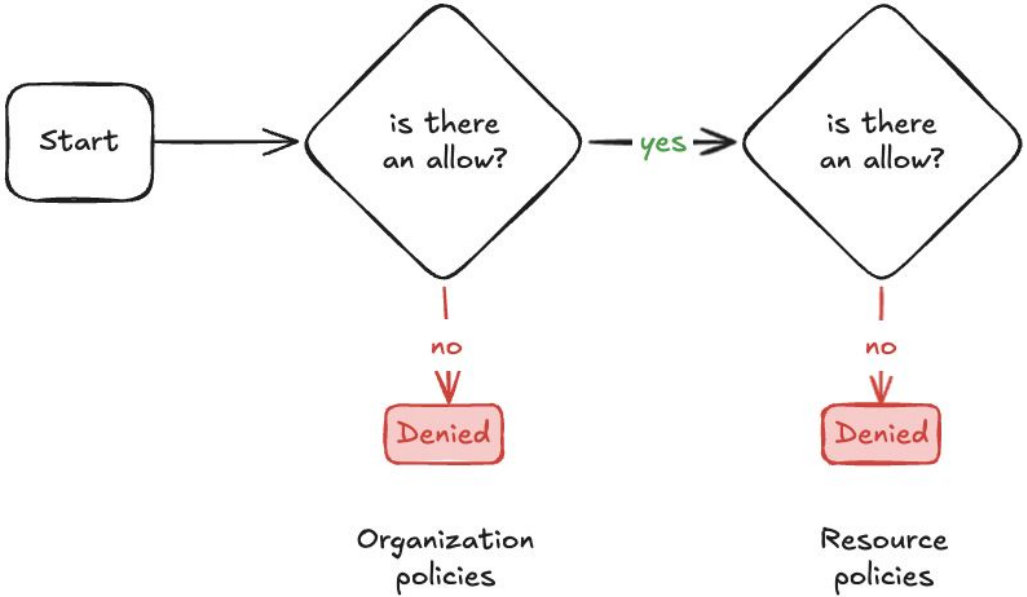
IAM Policies 101: Evaluation logic



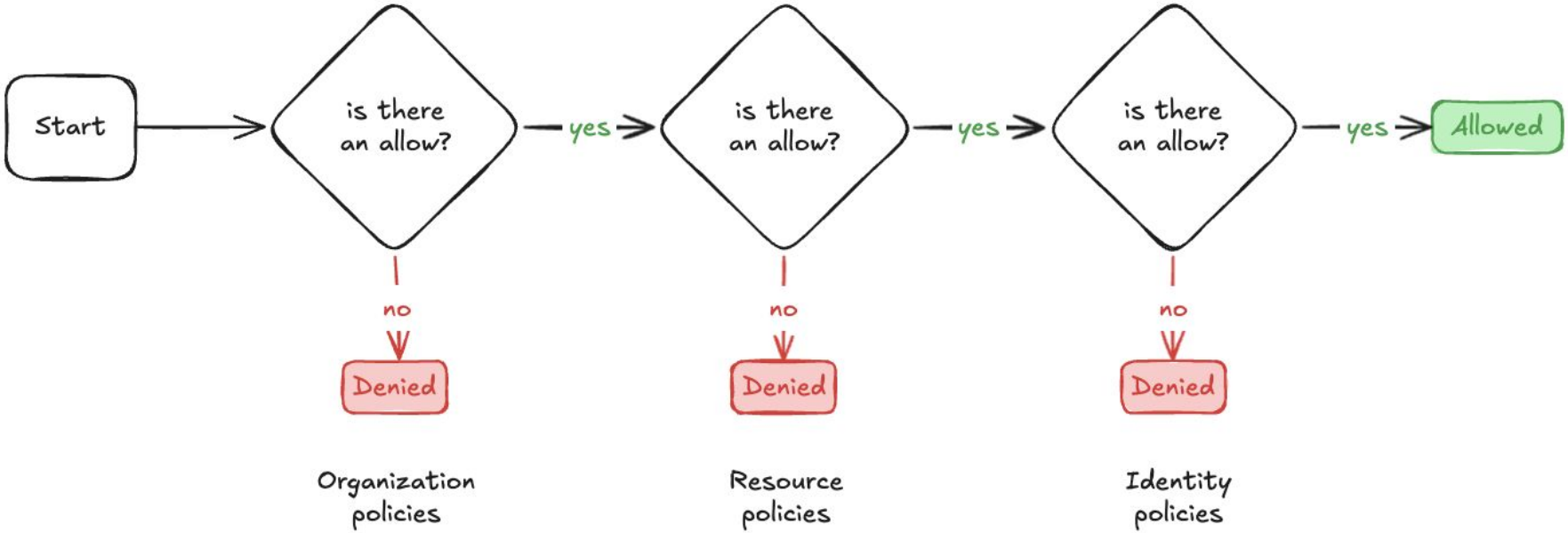
IAM Policies 101: Evaluation logic



IAM Policies 101: Evaluation logic



IAM Policies 101: Evaluation logic



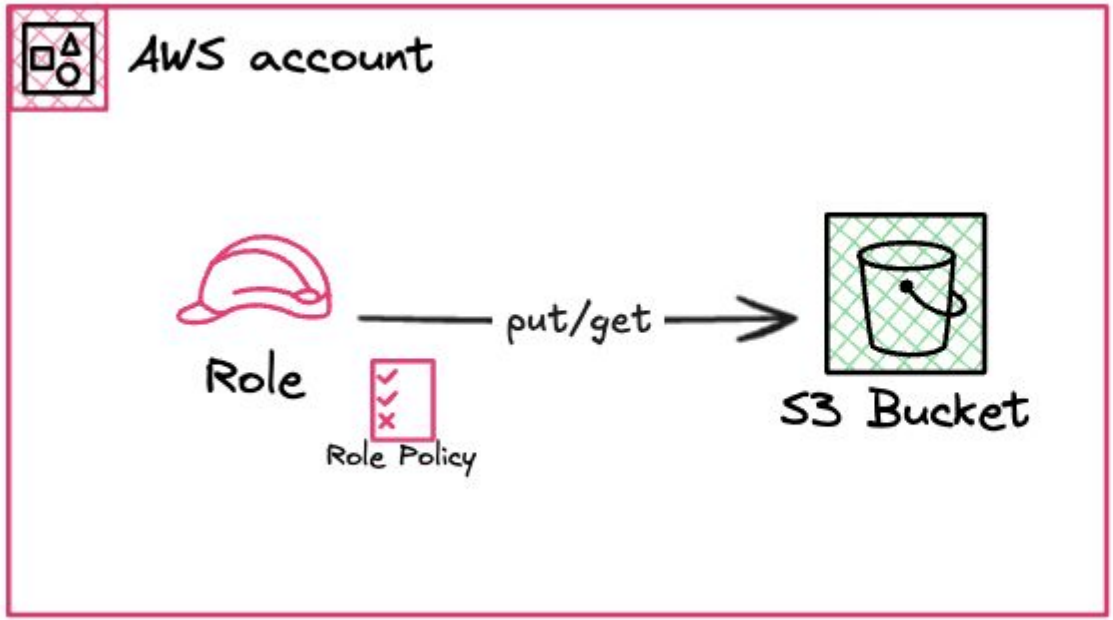
Case 1

S3



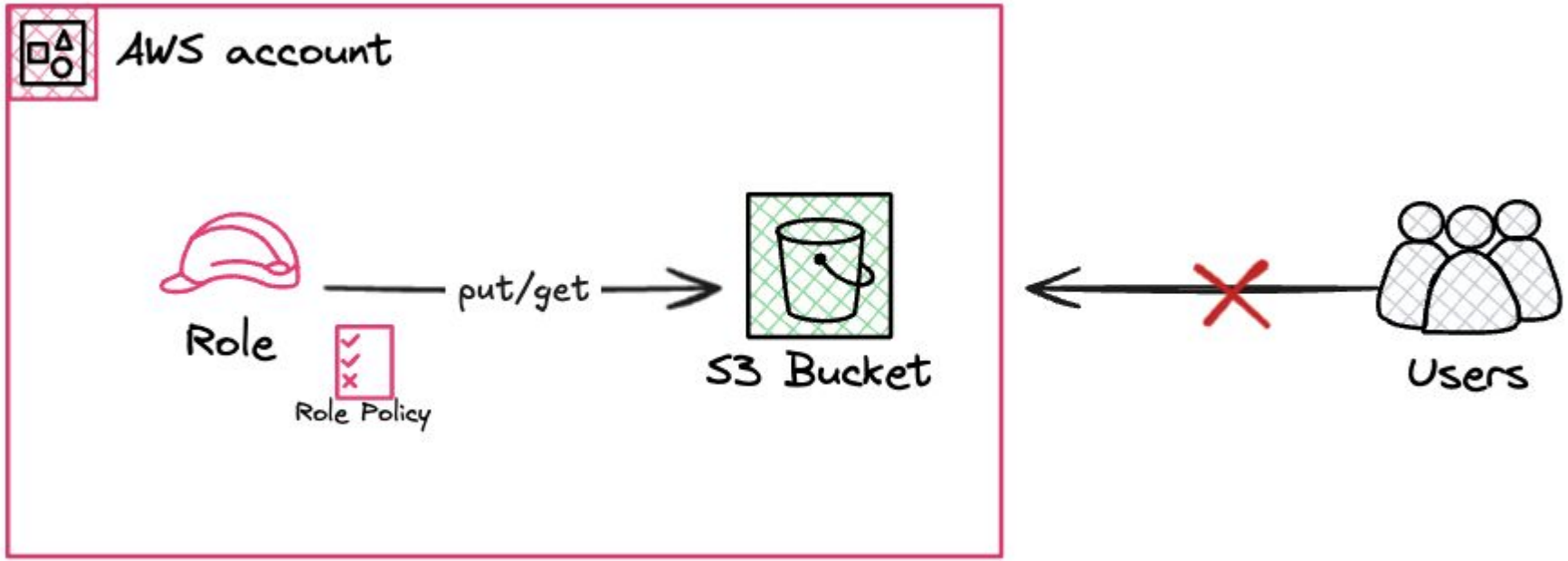
Case 1: AWS S3

Goal: Give Read and Write access to my application



Case 1: AWS S3

Goal: Give Read and Write access to my application



Case 1: AWS S3

How to give access to a S3 bucket:

- Make it public
- Add S3 permissions to our Identity
- Use S3 bucket policies
- Use S3 ACLs
- Use S3 Access points
- Use S3 Access grants



Case 1: AWS S3



S3 service has **165** available actions

Case 1: AWS S3.1

```
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Sid": "S3RWAccess",
            "Effect": "Allow",
            "Action": [
                "s3:*"
            ],
            "Resource": [
                "arn:aws:s3:::unicrons-invoices-bucket/*",
                "arn:aws:s3:::unicrons-invoices-bucket"
            ]
        }
    ]
}
```

- Delete it
- Make it public
- Disable logs
- Replicate it to another account
- Disable versioning *
- Disable retention protections



Case 1: AWS S3.1

```
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Sid": "S3RWAccess",
            "Effect": "Allow",
            "Action": [
                "s3:DeleteObject*",
                "s3:GetObject*",
                "s3:ListBucket",
                "s3:PutObject*"
            ],
            "Resource": [
                "arn:aws:s3:::unicrons-invoices-bucket/*",
                "arn:aws:s3:::unicrons-invoices-bucket"
            ]
        }
    ]
}
```

- ~~Delete it~~
- Make it public
- ~~Disable logs~~
- ~~Replicate it to another account~~
- Disable versioning *
- Disable retention protections



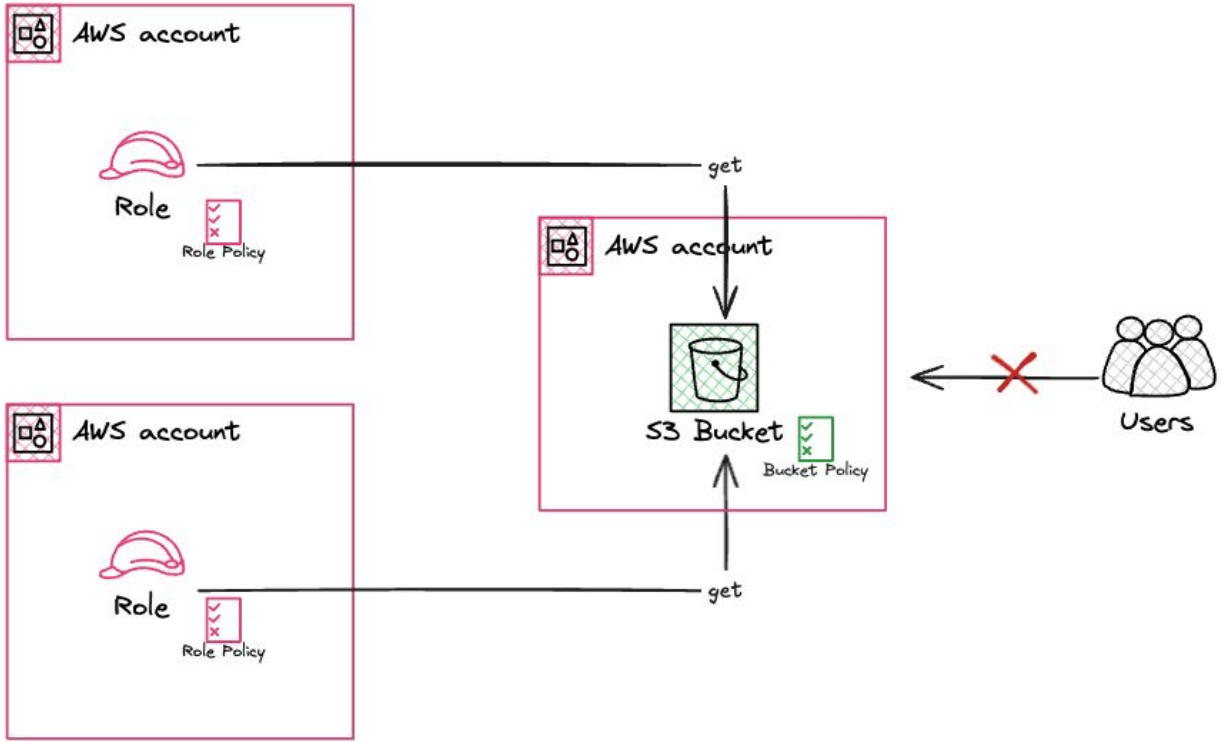
Case 1: AWS S3.1

```
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Sid": "S3RWAccess",
            "Effect": "Allow",
            "Action": [
                "s3:DeleteObject",
                "s3:GetObject",
                "s3:ListBucket",
                "s3:PutObject"
            ],
            "Resource": [
                "arn:aws:s3:::unicrons-invoices-bucket/*",
                "arn:aws:s3:::unicrons-invoices-bucket"
            ]
        }
    ]
}
```



Case 1: AWS S3.2

Goal: Give Read access to my applications in a different AWS account



Case 1: AWS S3.2

Goal: Give Read access to my applications in a different AWS account

Application Role Policy

```
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Sid": "S3RAccess",
            "Effect": "Allow",
            "Action": [
                "s3:GetObject",
                "s3:ListBucket"
            ],
            "Resource": [
                "arn:aws:s3:::unicrons-invoices-bucket/*",
                "arn:aws:s3:::unicrons-invoices-bucket"
            ]
        }
    ]
}
```

Case 1: AWS S3.2

Goal: Give Read access to my applications in a different AWS account

Bucket Policy

```
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Sid": "InvoicesPolicy",
            "Effect": "Allow",
            "Principal": {
                "AWS": "*"
            },
            "Action": [
                "s3:ListBucket",
                "s3:GetObject"
            ],
            "Resource": [
                "arn:aws:s3:::unicrons-invoices-bucket/*",
                "arn:aws:s3:::unicrons-invoices-bucket"
            ],
            "Condition": {
                "ForAllValues:StringEquals": {
                    "aws:PrincipalArn": [
                        "arn:aws:iam::111111111111:role/role-a",
                        "arn:aws:iam::222222222222:role/role-b"
                    ]
                }
            }
        }
    ]
}
```



Case 1: AWS S3.2

Goal: Give Read access to my applications in a different AWS account

Bucket Policy



Case 1: AWS S3.2

Using curl

```
❯ curl https://unicrons-invoices-bucket.s3-eu-west-1.amazonaws.com/
<?xml version="1.0" encoding="UTF-8"?>
<ListBucketResult xmlns="http://s3.amazonaws.com/doc/2006-03-01/"><Name>unicrons-invoices-bucket</Name><Prefix></Prefix><Marke
<MaxKeys>1000</MaxKeys><IsTruncated>false</IsTruncated><Contents><Key>flag.txt</Key><LastModified>2025-10-08T11:21:25.000Z</La
<ETag>&amp;quot;f03561fad2a21fdd527503ade34ecd01&amp;quot;</ETag><ChecksumAlgorithm>CRC32</ChecksumAlgorithm><ChecksumType>FULL_OBJECT
ype><Size>22</Size><Owner><ID>0567fcbf0f3738899a25d6257ca832f627d0b73fedbecd7da77f78c65b4ce953</ID><DisplayName>sbldevnet+demo
me></Owner><StorageClass>STANDARD</StorageClass></Contents></ListBucketResult>%
```

```
❯ curl https://unicrons-invoices-bucket.s3-eu-west-1.amazonaws.com/flag.txt
FLAG{Hello unicrons!}
```



Case 1: AWS S3.2

Using aws cli

```
❯ aws s3 ls s3://unicrons-invoices-bucket --no-sign-request
2025-10-08 13:21:25         22 flag.txt
```

```
❯ aws s3 cp s3://unicrons-invoices-bucket/flag.txt . --no-sign-request
download: s3://unicrons-invoices-bucket/flag.txt to ./flag.txt
```

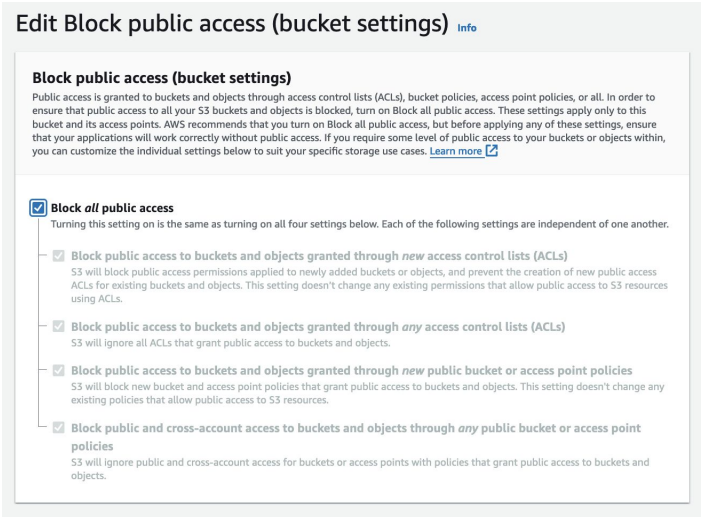


Case 1: AWS S3.2

Goal: Give Read access to my applications in a different AWS account

Recommended Bucket Policy

```
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Sid": "InvoicesPolicy",
            "Effect": "Allow",
            "Principal": {
                "AWS": "*"
            },
            "Action": [
                "s3:ListBucket",
                "s3:GetObject"
            ],
            "Resource": [
                "arn:aws:s3:::unicrons-invoices-bucket/*",
                "arn:aws:s3:::unicrons-invoices-bucket"
            ],
            "Condition": {
                "ArnEquals": {
                    "aws:PrincipalArn": [
                        "arn:aws:iam::111111111111:role/role-a",
                        "arn:aws:iam::222222222222:role/role-b"
                    ]
                },
                "StringEquals": {
                    "aws:PrincipalOrgID": [
                        "o-xxxxxxxxxxx"
                    ]
                }
            }
        }
    ]
}
```





Case 1: AWS S3.2

Bucket ARN
arn:aws:s3:::unicrons-invoices-bucket

Policy

```
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Sid": "BadPolicy",
            "Effect": "Allow",
            "Principal": {
                "AWS": "*"
            },
            "Action": [
                "s3:ListBucket",
                "s3:GetObject"
            ],
            "Resource": [
                "arn:aws:s3:::unicrons-invoices-bucket/*",
                "arn:aws:s3:::unicrons-invoices-bucket"
            ],
            "Condition": {
                "ForAllValues:StringEquals": {
                    "aws:PrincipalArn": [
                        "arn:aws:iam::111111111111:role/role-a",
                        "arn:aws:iam::222222222222:role/role-b"
                    ]
                }
            }
```

In recent years, AWS added:

- Block public access by default
- ACLs disabled by default
- Access Analyzer in bucket policy editor
- New ways of give access to S3

Security: 1 | Errors: 0 | Warnings: 0 | Suggestions: 0

Learn more about policy validation

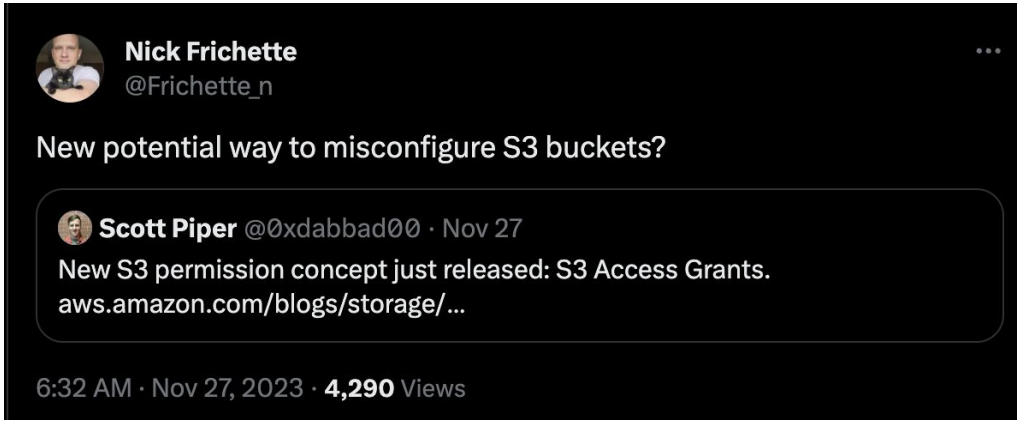
Search security warnings

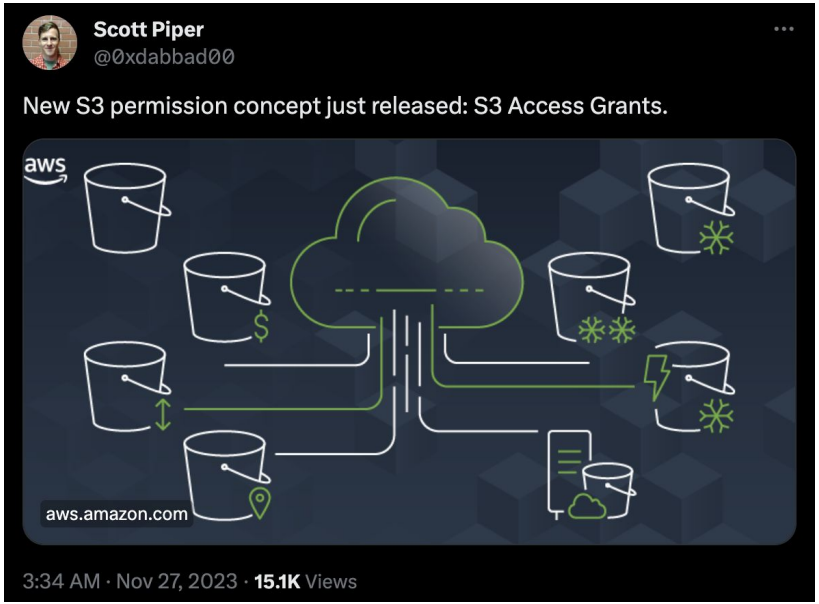
Ln 20, Col 5

Forallvalues With Single Valued Key: Using ForAllValues qualifier with the single-valued condition key aws:PrincipalArn can be overly permissive. We recommend that you remove ForAllValues:. Learn more

Case 1: AWS S3.2

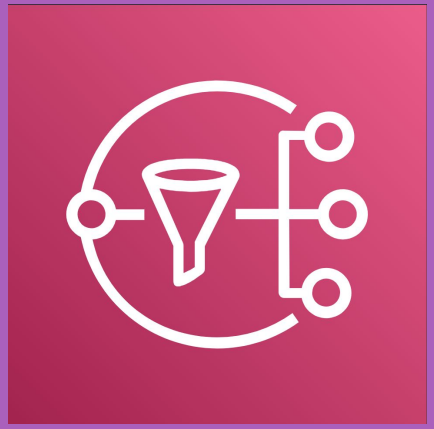
Did you say “new ways of give access to S3”?





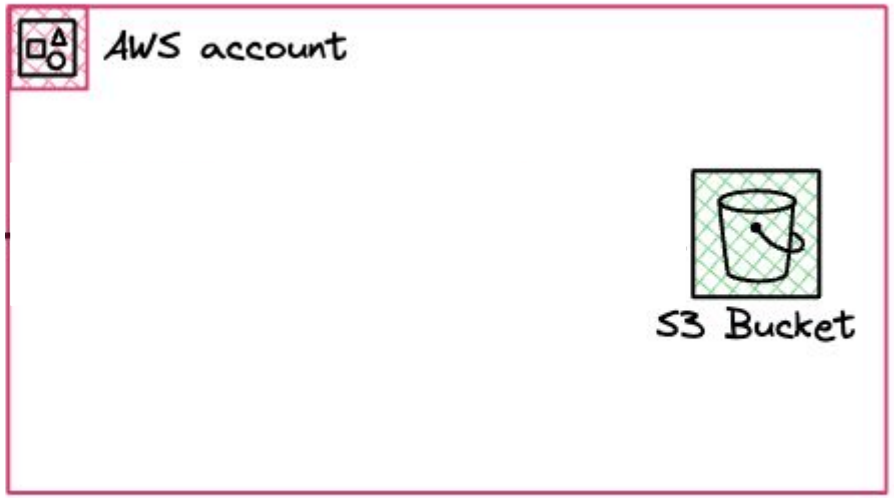
Case 2

SNS



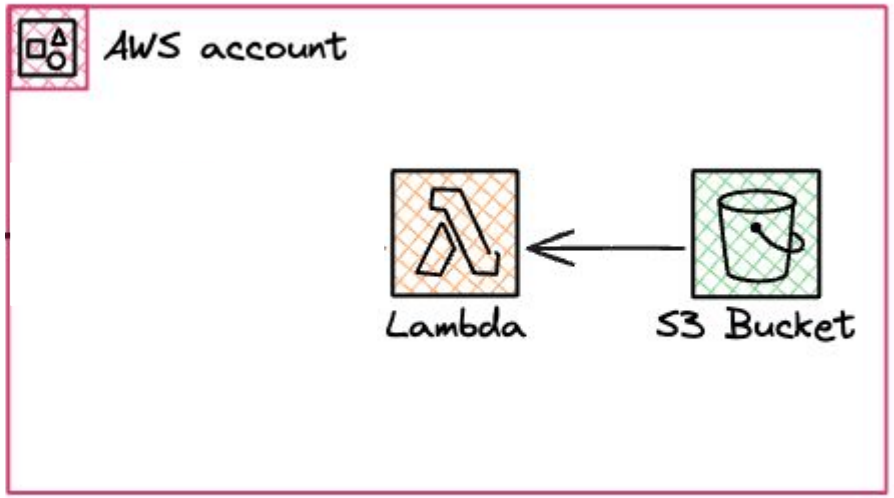
Case 2: AWS SNS

Goal: Allow subscription from *@unicrons.cloud users



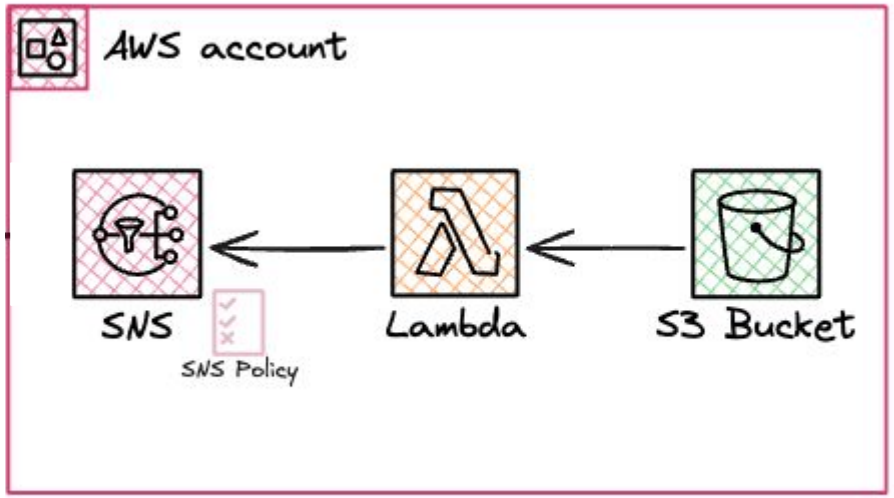
Case 2: AWS SNS

Goal: Allow subscription from *@unicrons.cloud users



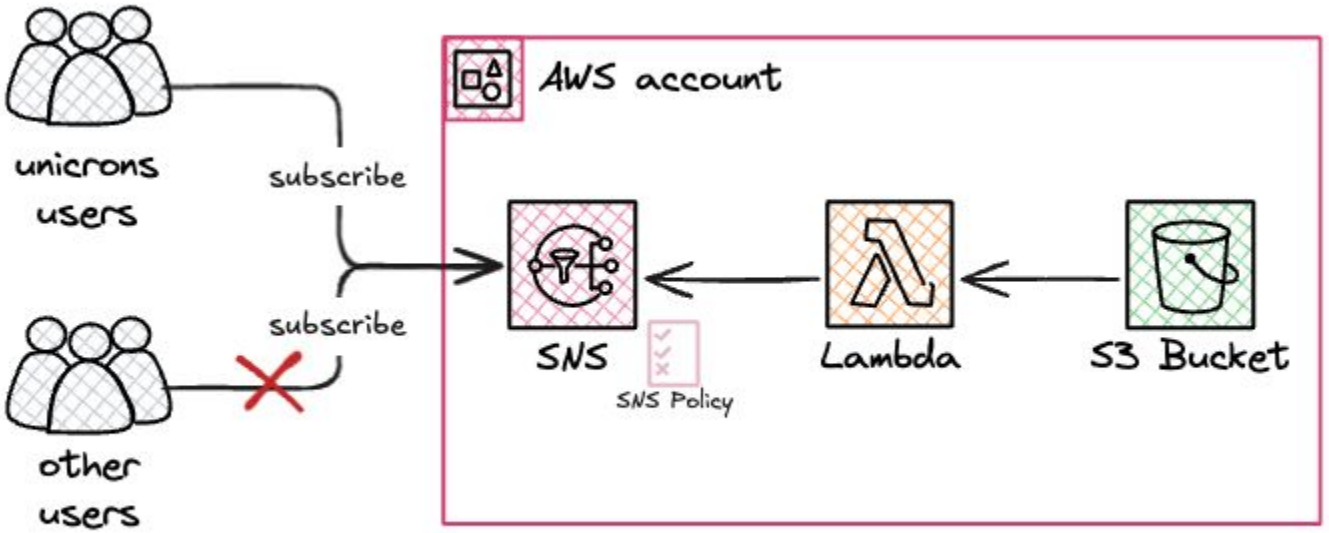
Case 2: AWS SNS

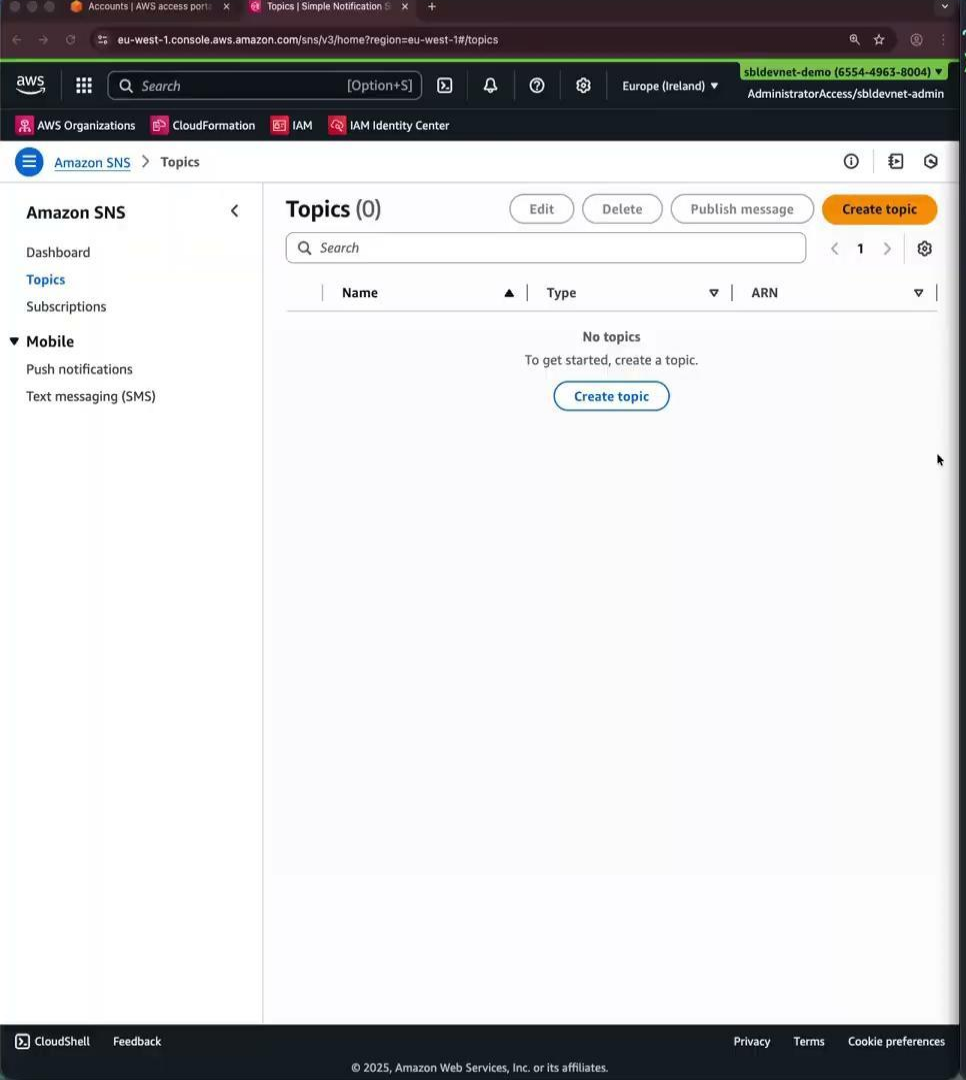
Goal: Allow subscription from *@unicrons.cloud users

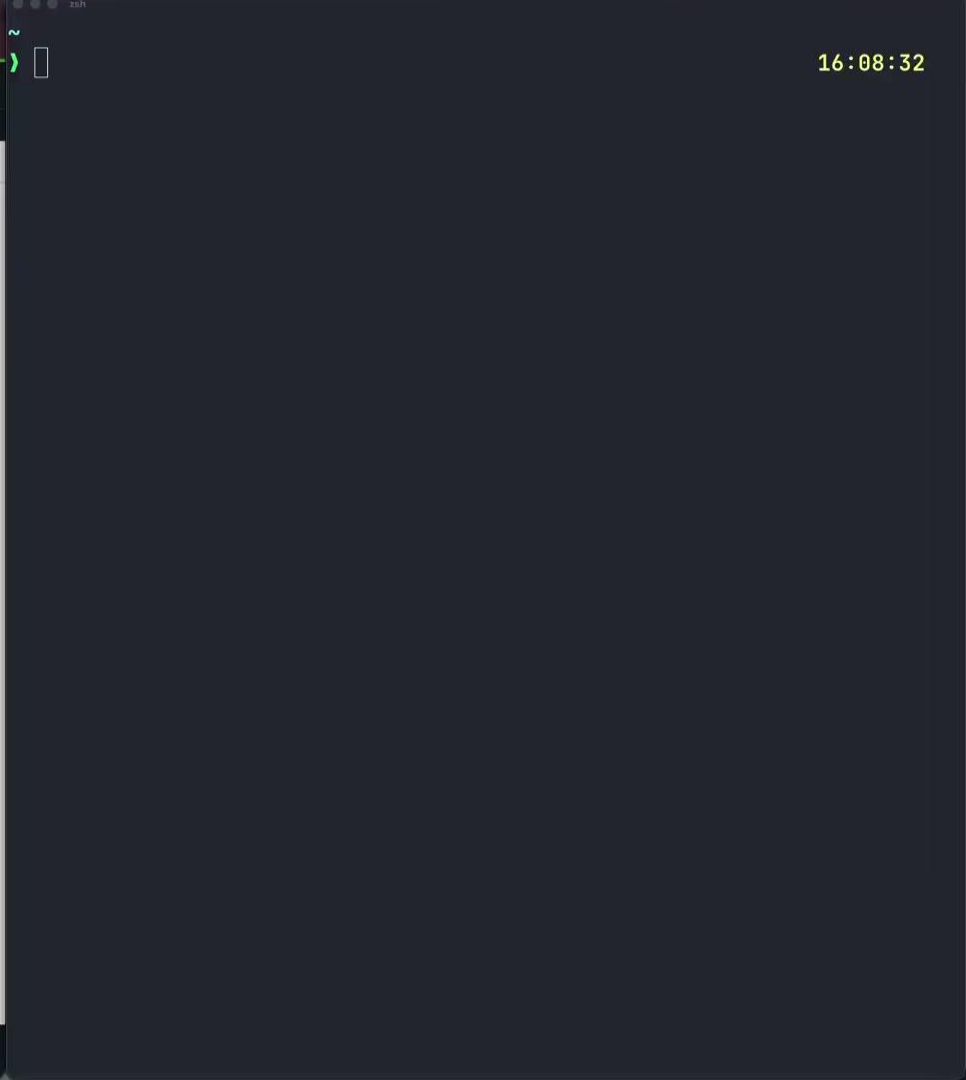


Case 2: AWS SNS

Goal: Allow subscription from *@unicrons.cloud users




Accounts | AWS access portal
Topics | Simple Notification Service
eu-west-1.console.aws.amazon.com/sns/v3/home?region=eu-west-1#/topics
aws
Search
[Option+S]
Europe (Ireland)
sbldevnet-demo (6554-4963-8004)
AdministratorAccess/sbldevnet-admin
AWS Organizations
CloudFormation
IAM
IAM Identity Center
Amazon SNS
Topics
Amazon SNS
Dashboard
Topics
Subscriptions
Mobile
Push notifications
Text messaging (SMS)
Topics (0)
Edit
Delete
Publish message
Create topic
Search
1
Name
Type
ARN
No topics
To get started, create a topic.
Create topic
CloudShell
Feedback
Privacy
Terms
Cookie preferences
© 2025, Amazon Web Services, Inc. or its affiliates.


16:08:32

unicrons-invoices

Details

Name
unicrons-invoices

ARN
arn:aws:sns:eu-west-1:123456789012:unicrons-invoices

Type
Standard

Display name
-

Topic owner
123456789012

Subscriptions | Access policy | Data protection policy | Delivery policy (HTTP/S) | Delivery status logging | Encryption | Tags

Access policy Info

This policy defines who can access your topic. By default, only the topic owner can publish or subscribe to the topic.



```
  {
    "Sid": "__console_sub_0",
    "Effect": "Allow",
    "Principal": {
      "AWS": "*"
    },
    "Action": "SNS:Subscribe",
    "Resource": "arn:aws:sns:eu-west-1:123456789012:unicrons-invoices",
    "Condition": {
      "StringLike": {
        "SNS:Endpoint": "*@unicrons.cloud"
      }
    }
  }
]
```

Case 2: AWS SNS

Example 3: Give users in the AWS account ability to subscribe to topics

In this example, we create a policy that grants access to the `Subscribe` action, with string matching conditions for the `sns:Protocol` and `sns:Endpoint` policy keys.

```
{
  "Statement": [{
    "Effect": "Allow",
    "Action": ["sns:Subscribe"],
    "Resource": "*",
    "Condition": {
      "StringLike": {
        "SNS:Endpoint": "*@example.com"
      },
      "StringEquals": {
        "sns:Protocol": "email"
      }
    }
  }]
}
```



Case 2: AWS SNS

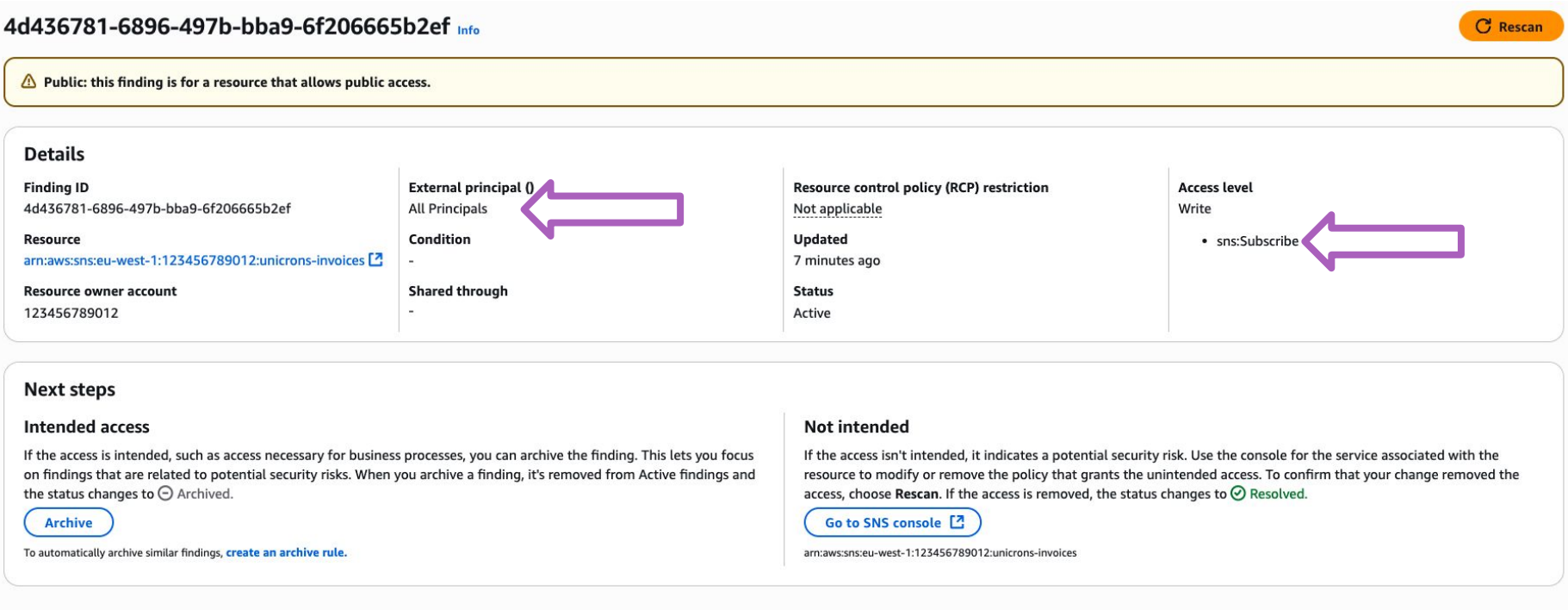
IAM Access Analyzer findings now support Amazon SNS topics and five other AWS resource types to help you identify public and cross-account access

Posted On: Oct 26, 2022

AWS Identity and Access Management (IAM) Access Analyzer now supports six additional resource types to help you identify public and cross-account access from outside your AWS account and organization. These six resource types include Amazon SNS topics, Amazon EBS volume snapshots, Amazon RDS DB snapshots, Amazon RDS DB cluster snapshots, Amazon ECR repositories, and Amazon EFS file systems. IAM Access Analyzer now analyzes resource policies, access control lists, and other access controls for these resources to make it easier for you to identify public, cross-account, and cross-organization access. These findings can help you adhere to the security best practice of least privilege and reduce unintended external access to your resources.



Case 2: AWS SNS



Case 3

Github Actions Federation



Case 3: Github Actions

The challenge started with a zip file that contained:

```
…/SOLVED/wardens/Wardens-Ruse-main
 ls -laR
drwxrwxr-x@    - Andoni.Alonso 15 jul 14:04 .github
.rw-rw-r--@ 1,4k Andoni.Alonso 15 jul 14:04 main.tf
.rw-rw-r--@  286 Andoni.Alonso 15 jul 14:04 README.md
.rw-rw-r--@ 174k Andoni.Alonso 15 jul 14:04 repo-visibility.png
.rw-rw-r--@ 465k Andoni.Alonso 15 jul 14:04 Warden.png

./.github:
drwxrwxr-x@ - Andoni.Alonso 15 jul 14:04 workflows

./.github/workflows:
.rw-rw-r--@ 829 Andoni.Alonso 10 ago 21:56 apply-prod.yaml

…/SOLVED/wardens/Wardens-Ruse-main
 cat README.md | grep TODO
TODO: Lock up once Haisha finishes setting up dc32-wardens-treasure-prod
and <https://d2azf0l1i0s26w.cloudfront.net/>.
```

Case 3: Github Actions

`main.tf` contained the creation of a role: `warden-production`

Apparently to be used for deploys, using Github Actions

Something suspicious?



```
resource "aws_iam_role" "warden" {
  name               = "warden-production"
  assume_role_policy = data.aws_iam_policy_document.warden-role.json
}

data "aws_iam_policy_document" "warden-role" {
  statement {
    actions = ["sts:AssumeRoleWithWebIdentity"]

    principals {
      identifiers = [aws_iam_openid_connect_provider.github.arn]
      type        = "Federated"
    }

    condition {
      test     = "StringEquals"
      values   = ["sts.amazonaws.com"]
      variable = "token.actions.githubusercontent.com:aud"
    }

    condition {
      test     = "StringLike"
      values   = ["repo:*/Wardens-Ruse:ref:refs/heads/endlessendurance"]
      variable = "token.actions.githubusercontent.com:sub"
    }

    condition {
      test     = "StringEquals"
      values   = ["private"]
      variable = "token.actions.githubusercontent.com:repository_visibility"
    }
  }
}
```

Case 3: Github Actions

```
    condition {
      test     = "StringEquals"
      values   = ["sts.amazonaws.com"]
      variable = "token.actions.githubusercontent.com:aud"
    }

    condition {
      test     = "StringLike"
      values   = ["repo:*/Wardens-Ruse:ref:refs/heads/endlessendurance"]
      variable = "token.actions.githubusercontent.com:sub"
    }

    condition {
      test     = "StringEquals"
      values   = ["private"]
      variable = "token.actions.githubusercontent.com:repository_visibility"
    }
  }
}
```



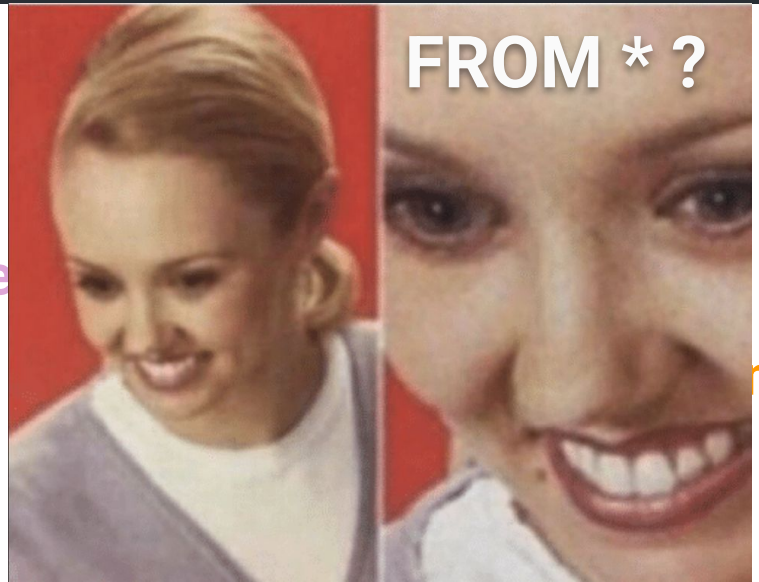
Case 3: Github Actions

```
condition {
  test     = "StringLike"
  values   = ["repo:*/Wardens-Ruse:ref:refs/heads/endlessendurance"]
  variable = "token.actions.githubusercontent.com:sub"
}
```

A repository

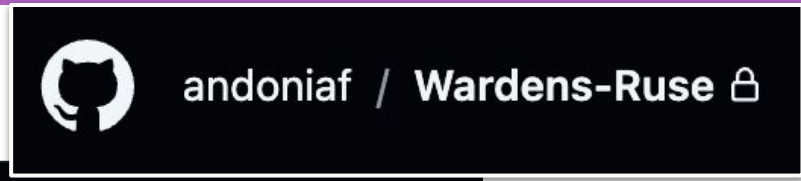
from “*”

name



ndlessendurance”

Case 3: Github Actions



← Deploy to Prod

Deploy to Prod #5

Summary

Jobs

Run Terraform

Run details

Usage

Workflow file

Run Terraform

failed 1 minute ago in 19s

- Set up job
- Run actions/checkout@v4
- Terraform action requires Node
- Setup Terraform
- Auth to AWS

```
1  ▸ Run aws-actions/configure-aws-credentials@v4
8  Assuming role with OIDC
9  Authenticated as assumedRoleId AROAZI2LHDAQJZPDN6EOB:GitHubActions
```

- Init

```
1  ▸ Run aws s3 ls --recursive s3://dc32-wardens-treasure-prod
11 2024-07-14 18:13:23         39 flag.txt
```

Key takeaways

- Review which service are you really using, block anything else
- IAM policies are NOT your friend
  - Do not blindly trust AWS managed policies
  - Avoid ‘*’ if possible
  - Use IAM Access Analyzer (if you can afford it)
- Enable Cloudtrail
- Alerting is your friend
- Stay up to date



Tools that can help you

Infra as Code



checkov

Runtime

PROWLER







Wanna play?

- https://bigiamchallenge.com/
- http://flaws.cloud/
- http://flaws2.cloud/
- https://awsiconquiz.com/



Thanks for coming!

@sbldevnet

sbldevnet

sbldevnet.com

@andoniaf_

andoniaf

andoniaf.unicrons.cloud

unicrons

@unicrons_cloud

unicrons.cloud



Session Rating:

